

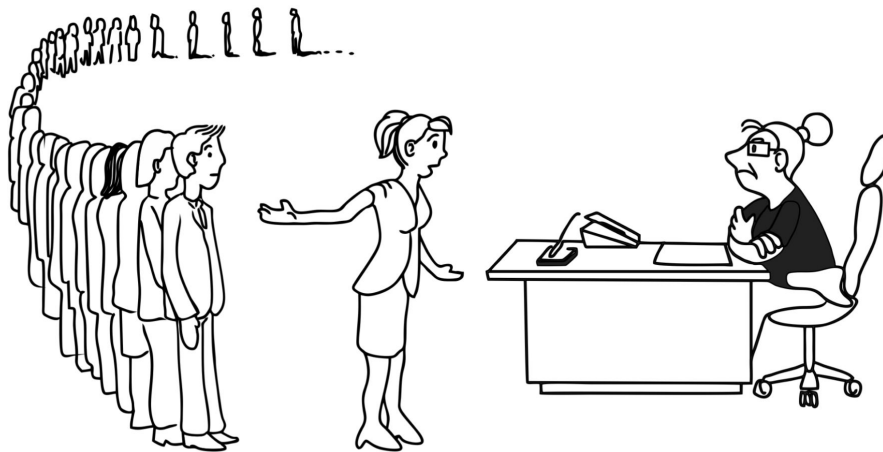
NP-Completeness

Motivação

- Até então nosso foco foi em encontrar algoritmos eficientes (polinomiais)
- Se não encontrarmos um algoritmo eficiente, temos duas possibilidades:
 - Ainda não descobrimos uma forma de resolver o problema em tempo polinomial
 - Não existe algoritmo de tempo polinomial para o problema
- Se não existir algoritmo eficiente, a insistência na busca será inútil
- Por isso é importante saber identificar que não existe, ou que é improvável que exista, algoritmo eficiente para o problema
- Classe de problemas NP-completos (NPC)
 - Mostrar que pertence a esta classe indica que é improvável a existência de algoritmo eficiente
 - Muitos problema pertencem a esta classe

Motivação

- Por que é “improvável” a existência de algoritmo eficiente em NPC?
 - Algoritmo polinomial p/ um problema em NPC produz algoritmo polinomial p/ todos em NP
 - Muitos pesquisadores já tentaram encontrar algoritmo polinomial para problemas em NPC
 - Porém, ninguém provou que não existe. Em aberto desde década de 70 (o famoso $P = NP?$)



“I can’t find an efficient algorithm, but neither can all these famous people.”

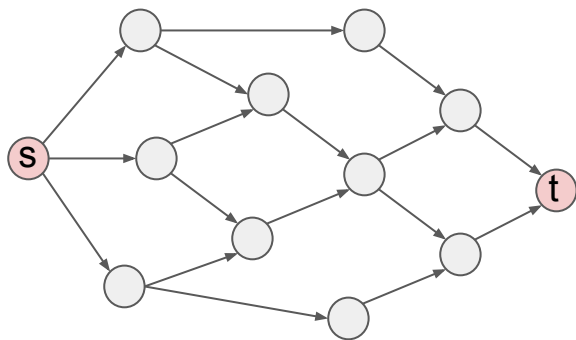
Fonte: [Garey & Johnson]

Sumário

- Problema de decisão x problema de otimização
- Classe de problemas P (polinomiais)
- Classe de problemas NP (polinomiais em máquinas não determinísticas)
 - Verificador polinomial
- Classe de problemas NPC (NP-completos)
 - Redução polinomial
- Passos para mostrar que um problema está em NPC
 - Exemplos de demonstrações

Problemas de decisão

- A teoria de NP-completude considera problemas de decisão
- Problemas cujo resultado é “Sim” ou “Não”
- Problemas de otimização possuem uma versão de decisão



Otimização: qual o tamanho do caminho mais curto entre os nós s e t no grafo G ?

Decisão: existe um caminho de tamanho no máximo k entre os nós s e t no grafo G ?

Problemas de decisão

- Solução para a versão de otimização permite resolver a versão de decisão
 - Ex.: Se o tamanho do caminho mais curto entre s e t vale 10, então sabemos decidir se existe caminho de tamanho no máximo k entre s e t , para qualquer valor de k .
- Então, existência de algoritmo polinomial para versão de otimização implica na existência algoritmo polinomial para a versão de decisão
 - Contrapositiva: inexistência de algoritmo polinomial para a versão de decisão implica na inexistência de algoritmo polinomial para a versão de otimização
- Ou seja, provar que não conhecemos algoritmo polinomial para a versão de decisão significa que também não conhecemos para a versão de otimização
 - A classe NPC é formada por problemas de decisão
 - Provar que um problema pertence a NPC implica que também não conhecemos algoritmo polinomial para a versão de otimização deste problema

Classe P

- Problemas de decisão que podem ser resolvidos em tempo polinomial
- Ex.: Existe caminho de tamanho no máximo k entre os nós s e t em um grafo com pesos não negativos nas arestas?
 - Podemos usar o algoritmo de Dijkstra (polinomial)

Classe NP

- Problemas de decisão que podem ser “verificados” em tempo polinomial
 - “Verificado” = possuir algoritmo verificador
- Algoritmo verificador
 - Entrada: entrada I do problema e um “certificado” c
 - Se I é uma instância “Sim”, então existe certificado c que fará o verificador retornar “Sim”.
 - Caso contrário, retorna “Não”.
- Observações:
 - O verificador pode retornar “Não” para uma instância “Sim” se receber um certificado “errado”
 - Quem projeta o verificador escolhe o que é um certificado “certo”
 - Geralmente usamos como certificado uma solução do problema

Classe NP

- Ex: Verificador para decidir se existe caminho de tamanho no máximo k entre os nós s e t em G .
 - A entrada do problema de decisão é a tupla (G,s,t,k)
 - Podemos usar como certificado um caminho C de tamanho no máximo k entre s e t .
 - Neste caso o verificador precisa apenas checar
 - Se C é um caminho válido de s até t
 - Se o custo do caminho C é no máximo k
 - Se passar nesta checagem, o verificador pode retornar “Sim”
 - Note que estas checagens podem ser feitas em tempo polinomial

Classe NP

- Todo problema em P pertence a NP
 - Verificador pode ignorar o certificado e simplesmente chamar o algoritmo polinomial
- O nome NP vem de “polinomial em máquina de Turing não determinística”
 - Máquinas não determinísticas são capazes de explorar vários caminhos (até a solução) simultaneamente
 - É como se ela fosse capaz de criar cópias de si mesma para explorar possibilidades
 - Infelizmente estas máquinas não existem na prática
- Existe demonstração de que um problema é verificado em tempo polinomial sse é resolvido em tempo polinomial em máquina não determinística

Redução polinomial

- Procedimento de tempo polinomial que resolve um problema X utilizando um número polinomial de chamadas a um algoritmo para o problema Y
 - Portanto, a existência de algoritmo polinomial para Y implica na existência de algoritmo polinomial para X

Classe NPC (NP-completos)

- Problemas em NP que possuem redução polinomial p/ todo problema em NP
 - Ou seja, para qualquer problema X em NPC e qualquer problema Y em NP
 - Existe procedimento de redução polinomial que decide Y utilizando algoritmo para X
 - Assim, a existência de um algoritmo polinomial para X implica na existência de algoritmo polinomial para todos os problemas em NP
- Concluimos que os problemas em NPC são os mais difíceis da classe NP

P = NP?

- Para provar que todos os problemas em NPC pertencem a P basta encontrar um algoritmo polinomial para algum problema em NPC
- Este problema está em aberto desde a década de 70
 - Ninguém encontrou algoritmo polinomial nem provou que não existe
- Ex. de impacto: criptografia utiliza problemas em NP explorando o fato de não dispormos (ainda) de algoritmo eficiente
- Questão filosófica: ainda não sabemos se verificar é mais difícil que resolver

Passos para mostrar que problema pertence a NPC

Desejamos mostrar que um problema de decisão Y pertence a NPC.

1. Mostre que Y pertence a NP
2. Identifique um problema X em NPC que facilite o passo seguinte
3. (Redução polinomial) Para qualquer instância I_x de X , mostre um procedimento polinomial que decide I_x fazendo um número polinomial de chamadas a um algoritmo para o problema Y .

Passos para mostrar que problema pertence a NPC

1. Mostre que Y pertence a NP

- Para isso forneça um verificador de tempo polinomial para Y.
 - Recebe como entrada uma instância I para Y e um certificado c
 - Você escolhe o conteúdo do certificado que lhe permita concluir que I é instância “Sim”
 - Mostre como usar este certificado para concluir que I é uma instância “Sim”
 - Mostre que este procedimento tem tempo polinomial

Passos para mostrar que problema pertence a NPC

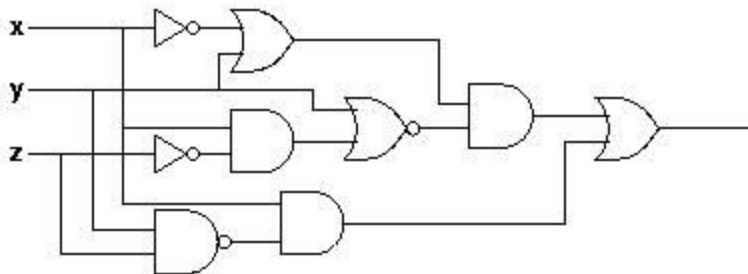
2. Identifique um problema X em NPC que facilite o passo seguinte
 - Procure em algum catálogo de problemas NPC, como o livro de Garey e Johnson

Passos para mostrar que problema pertence a NPC

3. (Redução polinomial) Para qualquer instância I_X de X , mostre um procedimento polinomial que decide I_X fazendo um número polinomial de chamadas a um algoritmo para o problema Y .
 - Caso com uma única chamada ao algoritmo para o problema Y :
 - Forneça algoritmo que transforma I_X em uma instância I_Y de Y
 - Mostre que este algoritmo tem tempo polinomial
 - Mostre que
 - Se I_X é uma instância “Sim” para X , então I_Y também é uma instância “Sim” para Y
 - Se I_X é uma instância “Não” para X , então I_Y também é uma instância “Não” para Y

Exemplos de problemas em NPC

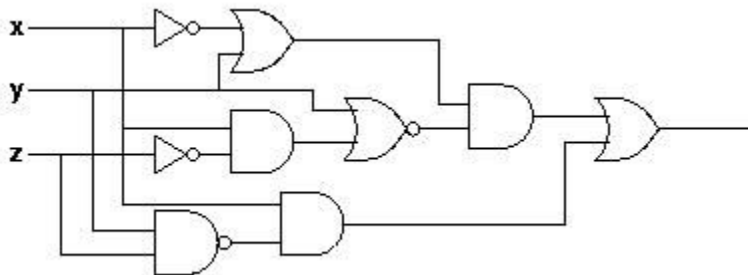
- Note que a demonstração utiliza outro problema em NPC
- Qual o primeiro problema provado NPC?
 - Satisfatibilidade de circuitos: existe atribuição de valor booleano (0 ou 1) para as entradas do circuito de modo que a saída do circuito seja 1?
 - Circuito é formado com portas lógicas AND, OR e NOT.



Exemplos de problemas em NPC

- Partindo da satisfatibilidade de circuitos podemos mostrar que a satisfatibilidade de fórmulas booleanas pertence a NPC
 - Satisfatibilidade de fórmulas booleanas (SAT): existe atribuição de valor lógico para as variáveis lógicas de modo que a fórmula tenha valor verdadeiro?
 - Um fórmula booleana possui variáveis lógicas e os operadores NOT, AND e OR.

$$\phi = ((\bar{x} \vee y) \wedge (\overline{y \vee (x \wedge \bar{z})}) \vee (x \wedge (\overline{y \wedge z})))$$



Exemplos de problemas em NPC

- **3-CNF-SAT**: satisfatibilidade de fórmulas lógicas na forma normal conjuntiva (CNF) onde cada cláusula possui 3 literais.
 - Literal: variável ou negação de variável
 - Cláusula: disjunção de literais
 - CNF: conjunção de cláusulas

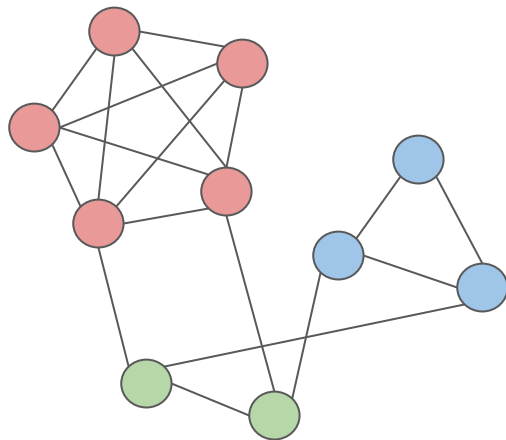
$$\phi = (x_1 \vee x_2 \vee x_3) \wedge (\overline{x_2} \vee x_3 \vee x_4) \wedge (x_2 \vee \overline{x_3} \vee \overline{x_4}) \wedge (x_1 \vee \overline{x_2} \vee x_4)$$

Exemplos de problemas em NPC

- 3-CNF-SAT pertence a NP
 - Certificado: atribuição de valor lógico para as variáveis que torna a fórmula verdadeira
 - Verificação em tempo polinomial
- Redução partindo do SAT
 - Existe procedimento de tempo polinomial que transforma uma fórmula qualquer φ_1 em uma fórmula equivalente φ_2 no formato 3-CNF
 - Portanto, φ_1 é satisfatível sse φ_2 é satisfatível
 - Podemos então decidir φ_1 utilizando um algoritmo para o 3-CNF-SAT aplicado em φ_2

Exemplos de problemas em NPC

- **Clique em grafos:** existe um subconjunto S de vértices de tamanho pelo menos k tal que existe aresta entre todo par de vértices em S ?



Exemplos de problemas em NPC

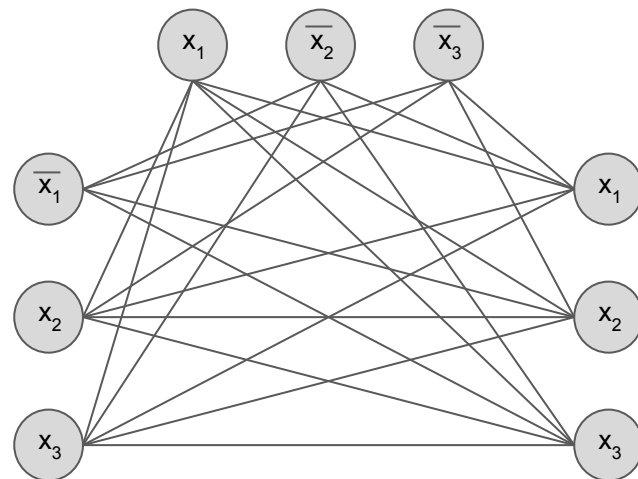
- Clique em grafos pertence a NP
 - Certificado: conjunto de vértices C
 - Verificar:
 - Elementos de C são vértices do grafo?
 - Tamanho de C é pelo menos k ?
 - Para cada par u e v de vértices em C , existe a aresta (u,v) no grafo?
 - Esta verificação pode ser realizada em tempo polinomial no tamanho do grafo

Exemplos de problemas em NPC

- Clique em grafos: redução partido do 3-CNF-SAT
 - Crie um nó para cada literal
 - Crie uma aresta para cada par de nós de cláusulas distintas que não representam uma variável e sua negação

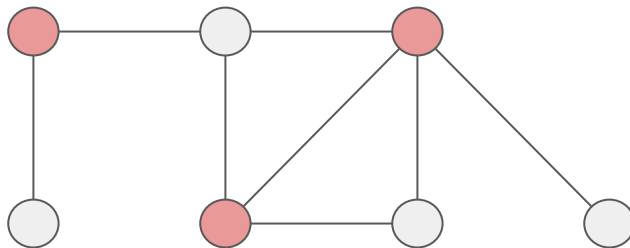
$$\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

- Se existe uma clique, então existe o que se pede e o problema é resolvido.
- Se não existe clique, então não existe o que se pede e o problema não é resolvido.
- Se existe uma clique, então existe o que se pede e o problema é resolvido.
- Se não existe clique, então não existe o que se pede e o problema não é resolvido.



Exemplos de problemas em NPC

- **Cobertura por vértices:** existe um subconjunto S de vértices de tamanho no máximo k tal que toda aresta incide em pelo menos um vértice em S ?

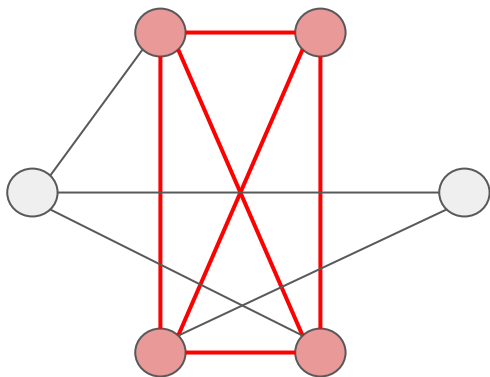


Exemplos de problemas em NPC

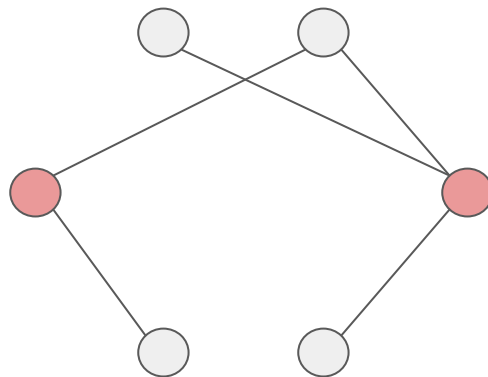
- Cobertura por vértices pertence a NP
 - Certificado: conjunto de vértices C
 - Verificar:
 - Elementos de C são vértices do grafo?
 - Tamanho de C é no máximo k ?
 - Para cada aresta (u,v) , pelo menos um dos nós u ou v pertence a C .
 - Esta verificação pode ser realizada em tempo polinomial no tamanho do grafo

Exemplos de problemas em NPC

- Cobertura por vértices: redução utilizando o problema clique
 - Grafo complemento de G : mesmos nós de G , e apenas as arestas que não ocorrem em G
 - C é uma cobertura no complemento de G sse $V-C$ é uma clique em G



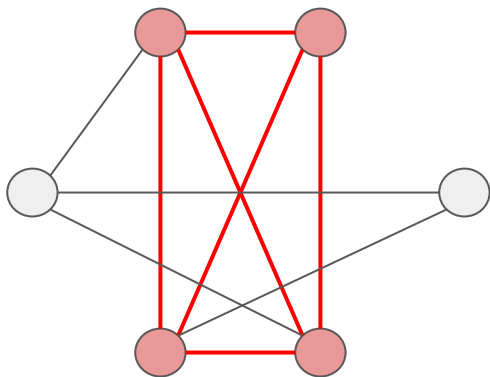
Clique em G



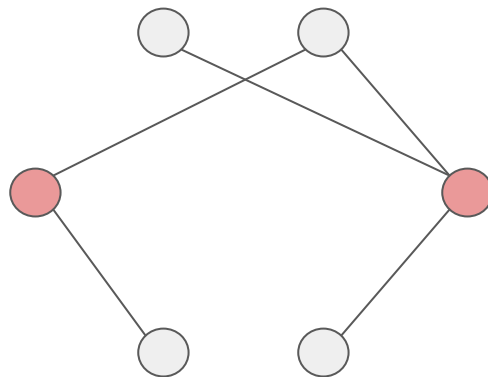
Cobertura no complemento de G

Exemplos de problemas em NPC

- Cobertura por vértices: redução utilizando o problema clique
 - Para saber se existe clique de tamanho pelo menos k em G , procuramos cobertura de tamanho no máximo $|V|-k$ no complemento de G .
 - A construção do grafo complemento é feita em tempo polinomial.



Clique em G



Cobertura no complemento de G

